# FullSend — Ground Truth Interview Document

> **How this was built**: Every claim below was verified directly from source code. Nothing is inferred from report documents. Where a report disagrees with code, the code wins.

## 0. Document Map — What Each Attached File Actually Is

| File | Purpose | Accuracy |
|---|---|---|
| `walkthrough.md` | Describes the *modularization* refactor — what changed and why. Written as a dev changelog. | ✅ Accurate — the changes it describes are all implemented in code. |
| `fullsend_system_design.md` | Architectural deep-dive written **before** the refactor. Describes the old monolithic `useFullSend.ts` and recommends the split as a "Suggested Improvement" (T1, T3, etc.). | ⚠️ Partially outdated — describes architecture that no longer exists. Improvements T1, T2, T3, T5, T6, T7, T8, T9, T10, T11, T12 are now implemented. |
| `fullsend_defense_report.md` | Amazon DA interview prep — SQL queries, STAR stories, analytical framing, mock Q&A. Mixes confirmed code facts with hypothetical instrumentation. | 🔶 Mixed — technical facts are accurate, but SQL schemas and instrumentation are clearly labeled hypothetical. |
| `interview_prep_fullsend.md` | 11 targeted adversarial interview Q&As grounded in code. | ✅ Accurate — every referenced file and line is correct for the **pre-refactor** version, but some references (e.g., "this is in `useFullSend.ts`") now point to the split hooks instead. |

## 1. Codebase: File-by-File Ground Truth

### Monorepo Root

**`package.json`** (root) — Bun workspace definition. Registers `backend`, `frontend`, and `shared` as workspace members.
**`bun.lock`** — workspace lockfile.
**`startup.md`** — brief developer quickstart (run backend, then frontend).

### `shared/` — `@fullsend/shared` Package

**`shared/package.json`**

```
{ "name": "@fullsend/shared", "main": "src/index.ts", "dependencies": { "zod": "^3.23.8" } }
```

A local workspace package consumed by both `backend` and `frontend` via `"@fullsend/shared": "workspace:*"`. No `peerjs`, no `cors`.

**`shared/src/constants.ts`** (7 lines)
Exports one constant: `CHUNK_SIZE = 65536` (64 KB) plus four string constants for message types (`FILE_META`, `FILE_CHUNK`, `FILE_DONE`, `FILE_ACK`).

**`shared/src/protocol.ts`** (163 lines)
The entire protocol schema layer. Contains:

- Zod schemas for every signaling message: `PidMessageSchema`, `NicknameUpdateSchema`, `ConnReqMessageSchema`, `ConnAcceptMessageSchema`, `ConnRejectMessageSchema`, `PeerListMessageSchema`.
- Zod schemas for every DataChannel message: `FileMetaSchema`, `FileChunkSchema`, `FileDoneSchema`, `FileAckSchema`.
- Derived TypeScript types via `z.infer<>`.
- Union types: `ClientMessage`, `ServerMessage`, `TransferMessage`.
- Factory objects `wsMsg` and `transferMsg` — typed builder functions so message construction is never inline object literals.
- `parseSignalingMessage()`, `parseClientMessage()`, `parseTransferMessage()` — each does `JSON.parse → switch on type → Zod.parse → return typed object or null`.

**`shared/src/index.ts`** (3 lines) — barrel re-export of protocol and constants.

## backend/ — Signaling Server

**`backend/package.json`**
Only dependency: `"@fullsend/shared": "workspace:*"`. Dev dependency: `@types/bun`. **No `peerjs`. No `cors`.** Both dead dependencies were removed.

**`backend/src/server.ts`** (89 lines)
The entire backend. A Bun native WebSocket server.

State:

```
const currentPeers = new Map<ServerWebSocket, string>();  // WS → peer ID
const nicknames   = new Map<string, string>();             // peer ID → nickname
```

**No `arr` array.** The old dual-state problem (T6) was fixed.

`Bun.serve()` config:

- `hostname: "0.0.0.0"`, `port: 8080`
- `fetch()`: upgrades all HTTP requests to WebSocket; returns 500 if upgrade fails
- `open(ws)`: sends `"Server connection open"` string (not a JSON protocol message — cosmetic)
- `message(ws, message)`: calls `parseClientMessage(message as string)` from shared package; handles 5 cases:
  - `"pid"`: `isNew` check prevents duplicate broadcast; stores in both Maps; calls `broadcast()`
  - `"nicknameUpdate"`: updates `nicknames` Map; calls `broadcast()`
  - `"connReq"` / `"connAccept"` / `"connReject"`: calls `forward(receiver, message, sender)`
- `close(ws)`: gets peer ID from `currentPeers`, deletes both Map entries, calls `broadcast()`

`broadcast()`: derives peer list from `currentPeers.values()` directly (no separate `arr`). Sends `{ type: "peerList", peers: [...] }` to every WebSocket.

`forward(targetId, message, senderId)`: O(n) linear scan through `currentPeers`. Enriches message with `senderNickname` from `nicknames`. Sends to first matching peer.

> **Known issues remaining in server.ts:**
>
> - `forward()` uses `Map<ServerWebSocket, string>` requiring linear scan for target lookup. Should be `Map<string, ServerWebSocket>` for O(1) forward.
> - `isNew` check ( `!Array.from(currentPeers.values()).includes(msg.id)` ) is also O(n).
> - No heartbeat/ping-pong — zombie connections from dead mobile clients will stay until next server restart.
> - No payload size limit (no `maxPayloadLength` in Bun.serve config).
> - No authentication — anyone who knows a peer ID can send `connReq` to them.

## `frontend/` — React SPA

**`frontend/src/index.ts`** — Bun HTTP server: serves `index.html` on `/*` , enables hot reload.

**`frontend/src/index.html`** (14 lines) — HTML shell. Title is **`FullSend`** (not "Bun + React" — this was fixed, A7 resolved).

**`frontend/src/frontend.tsx`** — React 18/19 root. `createRoot → <StrictMode> → <BrowserRouter> → <App />` .

**`frontend/src/App.tsx`** (28 lines)
Checks `localStorage.getItem("fullsend_nickname")` . If missing, renders `<NicknameModal>` . Otherwise renders `<PeerProvider nickname={nickname}> → <Routes>` with two routes: `/` (LobbyPage) and `/transfer/:peerId` (TransferPage).

**`frontend/src/lib/nameGen.ts`** (17 lines)
Uses `unique-names-generator` library. Generates names like `"swift-falcon-472"` . The shuffle logic randomly orders `[adjectives, animals]` arrays so the pattern varies. Appends a random 3-digit suffix.

> **Note**: The reports say the pattern is always "adjective-animal-NNN". The code shuffles the two dictionaries, so it can also produce "animal-adjective-NNN".

**`frontend/src/lib/utils.ts`** — `cn() = clsx + tailwind-merge` . Standard shadcn utility.

## Hooks

**`useFullSend.ts`** (88 lines) — The orchestrator. No longer contains any WebSocket or PeerJS logic. Composes `useSignaling` and `usePeerConnection` . Manages:

- `remotePeerId` state (the peer ID typed in the connect form)
- `myPeerId` state: starts empty, gets populated once `usePeerConnection` resolves the async PeerJS ID
- `currentNickname` state: a parallel nickname tracker in the orchestrator layer (the comment at line 45 admits `myNickname: initialNickname` could be stale — this is a known issue)
- `wrappedUpdateNickname` : updates both the signaling layer and the local state
- Exposes a flat API surface that pages consume via context

> **Live bug in `useFullSend.ts` line 45**: `usePeerConnection` is passed `myNickname: initialNickname` — the *original* value from when `App` mounted, not `currentNickname` . This means if the user changes their nickname, the `senderNickname` in `connReq` / `connAccept` / `connReject` will still show the old name until the page remounts. `wrappedUpdateNickname` tracks it locally but doesn't thread it into `usePeerConnection` .

**`useSignaling.ts`** (105 lines)
Manages WebSocket lifecycle.

- `currentNicknameRef` ref ensures reconnects always use the **latest** nickname (T5 fixed).
- `nicknameMap` is computed with `useMemo` keyed on `availablePeers` (T7 fixed).
- WS URL reads from `BUN_PUBLIC_WS_PORT` / `BUN_PUBLIC_WS_HOST` env vars with fallbacks (T8 fixed).

- Exponential backoff: starts at 1000ms, doubles on close, caps at 30000ms. Resets on successful open.
- `onmessage` only handles `peerList` and `connReq`. `connAccept` and `connReject` are handled via the `addSignalingListener` subscription system (described below), not by this switch.
- `addSignalingListener(type, handler)` — adds a filtered `message` event listener to the WS. Returns an unsubscribe function. This is how `usePeerConnection` hooks into signaling events without prop-drilling the WS.
- Exposes: `availablePeers`, `nicknameMap`, `pendingRequest`, `setPendingRequest`, `updateNickname`, `sendSignalingMessage`, `addSignalingListener`.

**`usePeerConnection.ts`** (168 lines)
Manages PeerJS and WebRTC DataConnection.

- `new Peer()` created on mount with **no config** — uses PeerJS default STUN (Google's public STUN) and PeerJS cloud for ICE negotiation. **No TURN server.**
- Listens for incoming connections via `peer.on("connection")`.
- Subscribes to `"connAccept"` and `"connReject"` signaling events via `addSignalingListener`.
- On `connAccept`: sets status → `"connected"`, calls `peerRef.current.connect(msg.sender)` to initiate DataChannel.
- On `connReject`: sets status → `"rejected"`.
- Timeouts: `connectTimerRef` (15s) fires if `dataConnRef.current` is still null after sending `connReq`. `acceptTimerRef` (10s) fires if `dataConnRef.current` is null after sending `connAccept`.
- `resolvedPeerId`: async value from `peer.on('open')`. Gets synced back up to `useFullSend` which then passes it to `useSignaling`.
- `sendToPeer`: thin wrapper over `dataConnRef.current?.send(data)`.
- Exposes: connection state, `handleConnect`, `acceptRequest`, `rejectRequest`, `disconnect`, `sendToPeer`.

**`useFileTransfer.ts`** (158 lines)
Owns the entire file transfer protocol.

- Receives `dataConn: DataConnection | null` as a prop.
- On `dataConn` change: clears `receiveStateRef`, registers `dataConn.on("data", handler)`, returns cleanup.
- Incoming message handler uses `parseTransferMessage(raw)` from `@fullsend/shared` — **fully type-safe and validated**.
  - `file-meta`: creates `FileReceiveState` entry in `receiveStateRef` (name, size, mimeType, totalChunks, empty Map for chunks).
  - `file-chunk`: stores `ArrayBuffer` by index in `state.chunks`. Increments `receivedBytes`.
  - `file-done`: iterates chunks 0..totalChunks-1, builds `Blob`, creates object URL, triggers auto-download via synthetic `<a>` click, sends `file-ack`, cleans up ref entry.
  - `file-ack`: updates `remoteTracker.receivedNames`.
- `sendFiles(files)`: iterates files, per-file sends `file-meta` then chunks then `file-done`. 5ms sleep between chunks. Updates `setProgress` every chunk.
- `myTracker.sentNames` updated after all files sent. `myTracker.receivedNames` updated on each `file-done`.

**`PeerContext.tsx`** (38 lines) — Creates `PeerContext`, provides `PeerProvider` (calls `useFullSend(nickname)`), exports `usePeer()` consumer.

## Pages

**`LobbyPage.tsx`** (125 lines)
Route `/`. Consumes `usePeer()`. Navigation effect: when `connectionStatus === "connected" && connectedPeerId && dataConn`, navigates to `/transfer/:connectedPeerId`. Renders: `PeerIdentity`, `PeerList`, `ConnectForm`, `ConnectionStatus`, `ConnectionDialog`. Inline nickname editing with `localStorage` persistence.

**`TransferPage.tsx`** (194 lines)
Route `/transfer/:peerId`. Consumes `usePeer()` and `useFileTransfer(dataConn)`.

- Session recovery (T12): reads `sessionStorage.item("fullsend_peer")` on mount. Writes on `nicknameMap` update.
- `connectionLost` state: triggered immediately if `dataConn` is null on mount or becomes null. Distinguishes "had connection then

lost" vs "never had connection" via `hadDataConnRef`.

- File queue management (T10): `queuedFiles` state, `removeFile()` function, per-file X button in UI.
- 2GB size guard (T10): `alert()` + filter before queuing.
- Drop zone: `onDrop` → `handleFiles`. File `<input multiple>`.
- Send button is a styled `<p>` element (not `<button>`).
- Progress bar: CSS width driven by `(progress.current / progress.total) * 100%`.

## 2. Tech Stack — What's Actually Used and Why

| Technology | Where Used | Real Reason |
|---|---|---|
| **Bun** | Backend runtime + frontend dev server | Native WebSocket server ( `Bun.serve` ), zero-config TypeScript execution, hot reload, native build. No `ws` , no `tsc` , no webpack. |
| **TypeScript (strict)** | Everywhere | `noUncheckedIndexedAccess` , `noImplicitOverride` , strict mode. |
| **Zod** | `@fullsend/shared/protocol.ts` | Runtime schema validation for both signaling and transfer messages. TypeScript types derived from Zod schemas — single source of truth. |
| **PeerJS** | `usePeerConnection.ts` | Abstracts ICE/SDP negotiation. Gives `Peer` , `DataConnection` API. Uses PeerJS cloud for ICE relay and Google's public STUN. |
| **React 19** | Frontend | Component model for connection state machine (idle/connecting/connected/rejected). |
| **React Router v7** | Frontend | Two routes: lobby ( `/` ) and transfer ( `/transfer/:peerId` ). |
| **TailwindCSS v4** | Frontend | OKLCH color system, dark mode tokens. `bun-plugin-tailwind` integrates with Bun build. |
| **shadcn/ui** | Frontend `components/ui/` | Copied-in primitives (Button, Input). Not a package dependency — fully customizable. |
| **unique-names-generator** | `lib/nameGen.ts` | Generates human-readable nicknames. |
| **Lucide React** | UI icons | `ArrowLeft` , `Upload` , `X` . |

## 3. Key Logic & Algorithms

### 3.1 WebSocket Signaling Design

The server uses `Map<ServerWebSocket, string>` (WS object as key, peer ID as value). This choice means:

- `close(ws)` handler can delete the entry with just `ws` — no need to look up the peer ID first.
- Downside: `forward()` requires O(n) iteration to find the target WS by peer ID. The inverse map `Map<string, ServerWebSocket>` would give O(1) forward. This is the biggest remaining performance issue.

### 3.2 Exponential Backoff Reconnect

```
retryDelay = 1000
onclose → setTimeout(connectWs, retryDelay); retryDelay = min(retryDelay * 2, 30000)
onopen  → retryDelay = 1000
```

Resets on success. Prevents thundering herd. `currentNicknameRef.current` ensures the re-registration `pid` message always carries the latest nickname.

### 3.3 Two-Phase Connection Handshake

```
User A: sendSignalingMessage(connReq) → start 15s timer
Server: forward to B, enrich with senderNickname
User B: sees ConnectionDialog → accept → sendSignalingMessage(connAccept) → start 10s timer
Server: forward to A
User A: on connAccept → peer.connect(B.id) → PeerJS ICE negotiation
         DataChannel opens → clearTimeout(15s timer) → setDataConn
User B: peer.on("connection") → DataChannel opens → clearTimeout(10s timer) → setDataConn
Both: navigate to /transfer/:peerId
```

Two separate timers cover two failure modes:

- 15s: B never responds (ignored, rejected, or offline)
- 10s: B accepted but ICE negotiation failed (NAT traversal failure)

### 3.4 Chunked File Transfer Protocol

```
Sender: file-meta (once) → [file-chunk × N, 5ms sleep each] → file-done
Receiver: accumulate chunks in Map<index, ArrayBuffer> → on file-done, assemble Blob → download → file-ack
Sender: on file-ack → update remoteTracker
```

**64KB chunk size**: Cross-browser WebRTC SCTP DataChannel safe upper bound.
**5ms sleep**: `await new Promise(r => setTimeout(r, 5))` — manual backpressure to prevent DataChannel buffer overflow. Creates hard ~12.8 MB/s throughput ceiling (64KB / 5ms = 12.8 MB/s).
**Chunk storage**: `Map<number, ArrayBuffer>` in a `useRef` — bypasses React render cycle. UI only updates on `setProgress` (per chunk) and `setMyTracker` (on file-done). This is correct design.
**Critical gap**: Missing chunks are silently skipped during reassembly ( `if (c) chunks.push(c)` ). Corrupted file is still delivered.

### 3.5 Session Recovery

On `TransferPage` mount:

1. Try `sessionStorage.getItem("fullsend_peer")` — returns `{ peerId, peerName }` if set.
2. If `peerId` matches the route param, use saved `peerName` (survives page refresh).
3. On `nicknameMap` update (live data), overwrite session storage with fresh name.
4. On `handleDisconnect` , clear session storage.

`connectionLost` state: set to `true` if `dataConn` is ever null. `hadDataConnRef.current` distinguishes "session interrupted" (never had conn) from "connection dropped" (had conn then lost).

## 4. Project Evolution — What Changed and Why

The `fullsend_system_design.md` was written against the original, pre-refactor codebase. The `walkthrough.md` documents what was actually implemented afterward. Here's the delta:

| Old State | Current State | Why Changed |
|---|---|---|
| Single 243-line `useFullSend.ts` god hook | Split into `useSignaling` + `usePeerConnection` + `useFileTransfer` + thin `useFullSend` orchestrator | Testability, separation of concerns, cognitive clarity |
| `arr: string[]` + `currentPeers` — dual state | `currentPeers` only, `broadcast()` derives list from `values()` | Eliminated dual-truth risk |
| `peerjs` + `cors` as backend dependencies | Removed — only `@fullsend/shared` | Dead code removal |
| Hardcoded `ws://localhost:8080` | Env vars `BUN_PUBLIC_WS_PORT` / `BUN_PUBLIC_WS_HOST` with fallbacks | Deployability |
| `nicknameMap` rebuilt on every render | `useMemo` keyed on `availablePeers` | Performance |
| `initialNickname` stale closure in reconnect | `currentNicknameRef` ref tracks live value | Bug fix |
| No session recovery on TransferPage | `sessionStorage` with "Session interrupted" UI | UX resilience |
| No file removal from queue | Per-file X button, `removeFile()` function | UX |
| No file size guard | 2GB `alert()` + filter in `handleFiles` | Crash prevention |
| No shared type package | `@fullsend/shared` Zod schemas used by both server and frontend | Type safety at protocol boundary |
| `TransferPage` handled its own DataChannel messages inline | Moved to `useFileTransfer` hook | Modularity |
| Page title "Bun + React" | "FullSend" | Correctness |

## 5. Claims vs. Reality — Discrepancy Register

**From `fullsend_system_design.md` (written pre-refactor):**

| Claim | Reality |
|---|---|
| Backend `package.json` lists `peerjs` and `cors` as dependencies | **False in current code.** Both removed. Backend only depends on `@fullsend/shared`. |
| `arr: PeerID[]` exists in server.ts state | **False in current code.** Removed. |
| The hook structure shows only `useFullSend.ts` and `PeerContext.tsx` in hooks/ | **Outdated.** Now has 5 files: `useSignaling.ts`, `usePeerConnection.ts`, `useFileTransfer.ts`, `useFullSend.ts`, `PeerContext.tsx`. |
| Line count table says "Core hook + context: 2 files, 281 lines" | **Outdated.** Hook layer is now 5 files, ~600+ lines combined. |
| T1–T12 presented as "suggested improvements" | **Most are already implemented.** T1, T2, T3, T5, T6, T7, T8, T9, T10, T11 (partial), T12 are done. |
| `nicknameMap` is rebuilt on every render (A3) | **Fixed.** `useMemo` in `useSignaling`. |
| `connectWs` closes over stale `initialNickname` (A5) | **Fixed.** `currentNicknameRef` pattern implemented. |
| `TransferPage.tsx:129` has `raw as any` cast | **Fixed.** `parseTransferMessage()` from shared package handles this safely. |
| Title is `Bun + React` (A7) | **Fixed.** `index.html` title is `FullSend`. |

### From `interview_prep_fullsend.md`:

| Claim | Reality |
|---|---|
| "File Reference: `frontend/src/hooks/useFileTransfer.ts` (Inside the `file-chunk` and `file-done` switch cases)" | ✅ Correct — this is where that logic lives. |
| "The 5ms delay is in `useFileTransfer.ts` inside the `sendFiles` for-loop" | ✅ Correct — line 136. |
| "Backend uses simple global variables ( `new Map()` )" | ✅ Correct. |
| "PeerJS defaults to reliable, ordered DataChannels" | ✅ Correct — SCTP reliable ordered by default. |
| "PeerJS defaults to a public Google STUN server" | ✅ Correct — no custom STUN/TURN configured. |
| "The code uses `connectTimerRef.current = setTimeout(...)` to transition to 'rejected' if DC doesn't open in 15s" | ✅ Correct — line 96-101 in `usePeerConnection.ts`. |

### From `fullsend_defense_report.md` (the most detailed doc):

| Claim | Reality |
|---|---|
| "90-second explanation" references `useSignaling` hook | ✅ Correct — this hook exists and does what the description says. |
| "The `@fullsend/shared` package defines CHUNK_SIZE in `shared/src/constants.ts` " | ✅ Correct — line 1 of constants.ts. |
| "Duplicate peer registration: `const isNew = !Array.from(currentPeers.values()).includes(msg.id)` " | ✅ Confirmed — line 28 of server.ts. |
| " `forward()` is O(n) linear scan" | ✅ Confirmed. Not fixed. |
| "Missing chunk: `if (c) chunks.push(c)` — gap confirmed" | ✅ Confirmed — line 72 of `useFileTransfer.ts` . |
| "The 5ms delay creates ~12.8 MB/s ceiling" | ✅ Mathematically correct: 64KB / 0.005s = 12.8 MB/s. |
| "Speed calculation is based on sender elapsed time, not actual receiver throughput" | ✅ Correct — `(Date.now() - startTime) / 1000` is sender-side. |
| Nickname pattern "adjective-animal-NNN" | **Partially wrong.** Code shuffles the two word lists, so it can be "animal-adjective-NNN" too. |
| "Connection request timeout: 15s, Accept timeout: 10s" | ✅ Correct — lines 96 and 115 of `usePeerConnection.ts` . |
| "open(ws) sends `'Server connection open'` " | ✅ Confirmed — line 16. This is a plain string, not a protocol message. |

### Resume Claims (from defense report's audit):

| Statement | Truth from Code |
|---|---|
| "no server or cloud storage in between" | ⚠️ **Misleading.** A Bun WebSocket signaling server IS required. File bytes bypass it, but the server exists. Safer: "file bytes never touch the server." |
| "end-to-end encrypted" | ⚠️ **Overstated.** WebRTC's DTLS/SRTP is browser-enforced. You didn't implement the encryption. Own this clearly. |
| "peer discovery" done by PeerJS | ❌ **Wrong.** Your custom WebSocket server handles peer discovery. PeerJS only handles ICE/SDP for the DataChannel itself. |
| "Built from scratch" | ✅ Defensible. You wrote the signaling server, protocol, chunking logic, handshake flow, and UI. PeerJS is a library dependency, same as React. |

## 6. Interview Q&A — Grounded in Code

### Architecture

**Q: Why is there both a WebSocket server and PeerJS?**

Two distinct responsibilities:

- **Your WebSocket server** (Bun, 89 lines): handles *who is online*. Tracks peer IDs and nicknames in memory. Routes `connReq/connAccept/connReject` control messages between specific peers. Never touches file data.
- **PeerJS**: once two peers have agreed to connect (via your signaling), PeerJS manages the WebRTC ICE negotiation — SDP offer/

answer exchange through PeerJS's own cloud. This is what actually opens the DataChannel. PeerJS uses Google's public STUN for candidate gathering.

You own layer 1. Layer 2 is a library. Be explicit about this distinction.

**Q: Why does the server use `Map<ServerWebSocket, string>` instead of `Map<string, ServerWebSocket>` ?**

The key is WS object so the `close(ws)` handler can do `currentPeers.delete(ws)` with O(1) without knowing the peer ID upfront. The tradeoff is that `forward()` needs O(n) iteration to find a target by peer ID. For the expected scale (small group session), this is fine. At scale, you'd reverse the map for O(1) routing and do `peerIdToWs.get(targetId).send(enriched)` .

**Q: Walk me through exactly what happens when I send a file.**

1. `TransferPage` : user drops file. `handleFiles()` checks `f.size > 2GB` — rejects with `alert()` if so. Queues valid files.
2. User clicks "send". `handleSend()` calls `sendFiles(queuedFiles)` from `useFileTransfer` .
3. `sendFiles()` : for each file, generates `crypto.randomUUID()` as `fileId` . Computes `totalChunks = Math.ceil(file.size / 65536)` .
4. Sends `file-meta` via `dataConn.send(transferMsg.meta(...))` .
5. Loop: `file.slice(start, end).arrayBuffer()` → `dataConn.send(transferMsg.chunk(...))` → `setProgress()` → `await new Promise(r => setTimeout(r, 5))` .
6. After all chunks: `dataConn.send(transferMsg.done(fileId))` . Stores `fileId → fileName` in `sentFilesRef` .
7. Receiver's `dataConn.on("data")` handler (in their `useFileTransfer` ): builds up `receiveStateRef.current.get(id).chunks` Map. On `file-done` : assembles `Blob` , creates object URL, triggers download, sends `file-ack` .
8. Sender receives `file-ack` : updates `remoteTracker.receivedNames` and cleans up `sentFilesRef` .

**Q: What is the role of the 5ms sleep?**

It's a manual backpressure mechanism. WebRTC DataChannels have an internal SCTP buffer. If you push thousands of 64KB chunks into `dataConn.send()` without yielding, the buffer overflows and the channel closes. The `await setTimeout(0...5ms)` yields to the JS event loop between chunks, letting the browser flush the buffer. The correct production approach is to monitor `dataConn.bufferedAmount` and wait for `bufferedamountlow` events — that's zero-waste vs. the fixed 5ms which imposes a hard 12.8 MB/s ceiling regardless of network speed.

**Q: Why does the receiver store chunks in a `Map<number, ArrayBuffer>` instead of an array?**

A `Map<index, ArrayBuffer>` is idempotent for duplicate chunks (overwrites at same key) and can handle out-of-order arrival — though in practice SCTP with `reliable: true` (PeerJS default) guarantees ordering. The `useRef` means no React renders happen during chunked receipt — only `setProgress` (sender side) and `setMyTracker` / `setRemoteTracker` (on completion) trigger renders.

## Trade-offs

**Q: What's the biggest performance problem in this codebase?**

The 5ms sleep is the most acute: it's a hard ceiling on throughput (~12.8 MB/s) that applies even on a gigabit LAN. For a 1GB file, that's a minimum of ~78 seconds sender-side CPU regardless of network speed. The fix is `bufferedAmount` -based backpressure — only send the next chunk when `dataConn.bufferedAmount < threshold` .

**Q: What are the scaling limits of the signaling server?**

Three problems:

1. **State is in-process memory** — no horizontal scaling. Behind a load balancer, two peers on different server instances can't see each other. Fix: Redis for shared state.
2. **O(N²) broadcast** — every `pid/nicknameUpdate/close` event sends the full peer list to every connected client. At 10K peers: 10K messages per event. Fix: rooms/channels limiting broadcast scope, or replace with query-based presence.
3. **O(n) forward** — linear scan to route a `connReq` . Fix: invert the map: `Map<string, ServerWebSocket>` keyed by peer ID.

**Q: What happens if a chunk is dropped mid-transfer?**

The reassembly loop is `for i = 0..totalChunks-1: if chunks.get(i) -> push` . A missing chunk is silently skipped. The Blob is assembled from fewer bytes than expected. The file is still downloaded — but corrupted. There is no checksum, no CRC, no retry mechanism. SCTP (WebRTC's transport) makes this unlikely within a session, but if the DataChannel closes and reopens, all state is lost anyway.

**Q: What happens when the user refreshes the page on `/transfer/:peerId` ?**

`dataConn` is gone — PeerJS connections don't survive page reload. `TransferPage` reads `sessionStorage.getItem("fullsend_peer")` to recover the peer name and shows a "Session interrupted — not connected" banner with a "Back to lobby" link. The session storage was written by the `nicknameMap` effect when the connection was live. This prevents a completely blank broken page, but full reconnection requires going back to the lobby and re-initiating the handshake.

**Q: What happens if the signaling server crashes while a DataChannel transfer is in progress?**

Nothing. The DataChannel is fully independent of the signaling server once established. The file transfer continues unaffected. If the signaling server restarts, `useSignaling` will reconnect with exponential backoff and re-register the peer. The ongoing DataChannel is unaffected.

**Q: Does FullSend work across symmetric NAT (corporate Wi-Fi, mobile hotspot)?**

It depends. PeerJS uses Google's public STUN server for ICE candidate gathering. STUN works for ~80% of consumer NATs. For symmetric NAT (common in enterprise), STUN fails — ICE candidates don't resolve to a publicly routable address. In that case, the connection attempt times out after 15 seconds ( `connect_timeout` ). There is **no TURN fallback** in the current code. To guarantee connectivity, you'd need to deploy Coturn and configure PeerJS with TURN credentials.

### Specific Code Questions

**Q: Why does `useFullSend.ts` have `myNickname: initialNickname` at line 45?**

This is an acknowledged bug in the code (the comment says "Note: This might be stale if updated"). `usePeerConnection` receives the nickname at construction time, and while `useFullSend` tracks `currentNickname` state and updates it, it doesn't pass that updated value to `usePeerConnection` 's props because hooks can't dynamically change their prop sources. The practical impact: if you change your nickname and then initiate a connection, the `connReq` message will show your *old* nickname as `senderNickname` . The correct fix is a ref inside `usePeerConnection` that's updated by a callback from `useFullSend` .

**Q: Why Bun over Node.js?**

For a pure I/O-bound WebSocket signaling server, Bun's native WebSocket implementation (backed by uWebSockets in C) eliminates the `ws` npm library entirely. `Bun.serve` handles WS natively. Zero-config TypeScript execution means no `ts-node` , no build step for the backend. On the frontend, `bun --hot` replaces webpack/vite. The risk is Bun's smaller ecosystem and relatively newer maturity. For a signaling server with one file of 89 lines, that risk is negligible.

## 7. STAR Stories — Based on Actual Engineering Decisions

### STAR 1 — Designing the Protocol Boundary

**Situation**: The original FullSend had both the signaling server ( `server.ts` ) and the frontend ( `useFullSend.ts` ) constructing raw JSON objects inline — `ws.send(JSON.stringify({ type: "connReq", sender: ..., receiver: ... }))` at multiple call sites, with no shared type checking. If a field name changed on one side, the other side would silently break at runtime.

**Task**: Create a single source of truth for all protocol messages that both the Bun backend and the React frontend could import, enforced at compile time *and* runtime.

**Action**: Created `@fullsend/shared` as a Bun workspace package. Defined every signaling and DataChannel message as a Zod schema (e.g., `ConnReqMessageSchema` ), derived TypeScript types via `z.infer<>` , and built typed factory objects ( `wsMsg.connReq(...)` , `transferMsg.chunk(...)` ) so no code site constructs raw objects. Both `server.ts` and `useSignaling.ts`

import from `@fullsend/shared`. If any message field changes, TypeScript compile fails everywhere it's constructed or consumed.

**Result**: Protocol changes are now compile-time errors on both sides simultaneously. The `parseClientMessage` / `parseTransferMessage` functions validate at runtime and return `null` on malformed input — no silent field-access crashes. The backend's only dependency is `@fullsend/shared`.

## STAR 2 — The DataChannel Buffer Overflow Problem

**Situation**: Initial file transfer implementation looped through all chunks and called `dataConn.send()` synchronously for each. On large files (>100MB), the DataChannel would silently close or throw.

**Task**: Make large file transfers reliable without rewriting the entire transfer layer.

**Action**: Diagnosed the issue as SCTP buffer overflow — the DataChannel has a finite internal buffer, and saturating it causes the channel to fail. Researched the correct fix (`bufferedAmount` / `bufferedamountlow` event monitoring), but chose the expedient approach: `await new Promise(r => setTimeout(r, 5))` between each chunk. This yields to the JS event loop, allowing the browser to flush the buffer. Documented the trade-off: the 5ms sleep creates a hard ~12.8 MB/s ceiling. Also stored the awareness of the proper fix (`bufferedAmountLowThreshold` pattern) as a known next step.

**Result**: Large file transfers are reliable. The trade-off (throughput ceiling) is conscious and documented. On slow networks this ceiling doesn't matter; on fast networks it's the bottleneck. A real production version would use backpressure.

## STAR 3 — Breaking Up the God Hook

**Situation**: The original `useFullSend.ts` was ~243 lines managing WebSocket connection logic, PeerJS instance lifecycle, connection FSM, file transfer chunking, progress tracking, and ACK handling all in one closure. Adding the session recovery feature or debugging reconnect timing meant reading through all of it.

**Task**: Refactor without breaking the existing context API that `LobbyPage` and `TransferPage` consumed.

**Action**: Identified three independent domains — signaling (WebSocket), peer connection (PeerJS/WebRTC), and file transfer (DataChannel protocol). Extracted each into a dedicated hook. Designed `addSignalingListener(type, handler)` as a pub/sub bridge so `usePeerConnection` could subscribe to `connAccept` / `connReject` events from the WebSocket without the orchestrator needing to thread callbacks through props. Made `useFullSend` a thin composition layer that kept the same external API (so `PeerContext.tsx` and all consumers required zero changes).

**Result**: Each hook is independently readable (~100-168 lines each). `useFileTransfer` can now be tested with a mock `DataConnection` object. `useSignaling` can be tested with a mock WebSocket. The orchestrator is 88 lines. File size guide: `useFileTransfer.ts` — 158 lines, `usePeerConnection.ts` — 168 lines, `useSignaling.ts` — 105 lines, `useFullSend.ts` — 88 lines.

## STAR 4 — Fixing the Stale Closure in WebSocket Reconnect

**Situation**: The original reconnect logic closed over `initialNickname` (the value at hook initialization time). If a user renamed themselves and the WebSocket reconnected (due to a network blip), the server would see the old nickname — the user would appear in the peer list under their previous name with no indication anything was wrong.

**Task**: Ensure that WebSocket reconnect always registers the user with their *current* nickname, not the one they had when the page first loaded.

**Action**: Added `currentNicknameRef = useRef(initialNickname)` in `useSignaling`. On `ws.onopen`, the `pid` message uses `currentNicknameRef.current` instead of the closed-over `initialNickname`. The `updateNickname` function updates both the ref (for reconnect) and sends a `nicknameUpdate` message to the live server (for immediate effect without reconnect).

**Result**: Nickname state is consistent across reconnects. The ref pattern (mutable value accessible inside the closure without re-creating the closure) is the standard React solution for "I need a live value in a stale closure."

### STAR 5 — Session Recovery on TransferPage

**Situation**: If a user on `/transfer/:peerId` accidentally hit browser refresh, they'd land on the TransferPage with `dataConn = null` (PeerJS doesn't survive reload), `nicknameMap` empty (WebSocket not yet connected), and no way to know who they were connected to. The page would show "Connected to Unknown" and immediately show the "connection lost" banner with no context.

**Task**: Make the refresh experience recoverable rather than broken — show the user what happened and give them a clear path forward.

**Action**: Before navigating to `/transfer/:peerId`, `TransferPage` writes `{ peerId, peerName }` to `sessionStorage`. On mount, it reads this back. If `dataConn` is null and `hadDataConnRef.current` is false (never had a connection), it shows "Session interrupted — not connected" instead of "Connection lost." Either way, a "Back to lobby to reconnect" link is shown. When live `nicknameMap` data arrives (WebSocket reconnects), the displayed name is refreshed and session storage is updated.

**Result**: A page refresh on the transfer screen now gives a comprehensible error state with actionable next steps instead of a silent broken UI. The session storage cleanup happens in `handleDisconnect` so no stale entries persist after a deliberate disconnect.

## 8. Known Live Issues (be honest about these)

| Issue | Location | Severity | Fix |
| --- | --- | --- | --- |
| `myNickname` is stale in `usePeerConnection` | `useFullSend.ts:45` | Low — only affects `senderNickname` display, not connection logic | Thread a `currentNicknameRef` into `usePeerConnection` |
| `forward()` is O(n) | `server.ts:78` | Low at current scale, problematic at 1K+ peers | Reverse the map: `Map<string, ServerWebSocket>` |
| `isNew` check is O(n) | `server.ts:28` | Same as above | Same map reversal |
| Missing chunk silent corruption | `useFileTransfer.ts:72` | High in unreliable networks (rare with SCTP reliable mode) | Per-chunk checksum or `totalChunks` vs received count check |
| 5ms throughput ceiling | `useFileTransfer.ts:136` | Medium — ~12.8 MB/s cap on fast networks | Replace with `bufferedAmount` backpressure |
| No heartbeat for zombie connections | `server.ts` | Medium — mobile clients on cell can ghost | Ping/pong every 30s, evict after 2 misses |
| No TURN server | `usePeerConnection.ts:37` | High in enterprise/ symmetric NAT environments | Self-host Coturn |
| 2GB check is UI-only | `TransferPage.tsx:58` | Low — bypassable via console | Implement check inside `sendFiles()` in `useFileTransfer` |
| No payload size limit on WS server | `server.ts` | Medium — DoS vector | Add `maxPayloadLength` to `Bun.serve` config |
| `file-ack` updates `myTracker.sentNames` via `remoteTracker` naming | `useFileTransfer.ts:94` | Minor UX confusion | The variable naming is inverted from the perspective of the receiving side |